

Étude de Cas : Développement d'une Application de Gestion des Stocks

1. Contexte du Projet

Une entreprise spécialisée dans la distribution de produits électroniques (ordinateurs, téléphones, accessoires), gère un stock réparti entre un entrepôt central et cinq magasins physiques. L'équipe logistique rencontre plusieurs problèmes :

- Des ruptures fréquentes sur des articles populaires comme les **iPhones** et les **cartes graphiques**.
- Des surstocks sur certains produits moins demandés, comme des **modèles d'ordinateurs obsolètes**.
- Un suivi manuel des stocks via **Excel**, entraînant des erreurs de saisie.

Pour répondre à ces défis, l'entreprise souhaite développer une **application centralisée de gestion des stocks**.

2. Étape "Délimiter"

2.1 Recenser les documents existants

L'équipe projet collecte les documents suivants :

- **Rapports d'inventaire des 6 derniers mois** : montrent un taux d'erreur de 15 % entre les stocks physiques et théoriques.
- **Historique des ventes** : indique que les téléphones haut de gamme partent en 3 jours en moyenne, mais les stocks ne sont pas toujours ajustés en conséquence.
- **Procédures internes** : révèlent qu'un seul employé est chargé de la mise à jour des stocks, ce qui ralentit la réactivité.

Document	Contenu	Problème identifié
Rapports d'inventaire (6 derniers mois)	Disparités entre le stock physique et le stock théorique (écart moyen de 15 %)	Gestion manuelle source d'erreurs
Historique des ventes	Produits les plus vendus : iPhones, cartes graphiques	Ruptures fréquentes sur ces produits
Procédures internes	Mise à jour des stocks par un seul employé via Excel	Processus lent et peu fiable
Rapports de commandes fournisseurs	Réapprovisionnement manuel basé sur des estimations	Délai de livraison parfois trop long

2.2 Interroger les personnes concernées

Acteur interrogé	Problème identifié	Besoin exprimé
Marc (Responsable logistique)	Suivi manuel des stocks via Excel, erreurs fréquentes	Automatiser la mise à jour des stocks en temps réel
Sophie (Gérante de magasin)	Ruptures de stock fréquentes sur produits populaires	Alerte automatique pour le réapprovisionnement
Ali (Développeur ERP)	Pas d'API sur le système actuel, complexité d'intégration	Accès direct à la base de données pour éviter doublons
Direction Générale	Manque de visibilité sur la gestion des stocks	Tableaux de bord et reporting en temps réel

2.3 Résultats de l'étape "Délimiter"

Énoncé des besoins

- L'application doit permettre :
Un **suivi des stocks en temps réel** dans chaque magasin et à l'entrepôt.
- L'**automatisation des alertes** pour le réapprovisionnement.
- Une **synchronisation avec l'ERP** de l'entreprise.

Périmètre et limites du projet

- **Inclus** : stocks des magasins et entrepôt, gestion des commandes.
- **Exclus** : stocks des fournisseurs externes, comptabilité financière.

Acteurs impliqués

- **Chef de projet** (coordination)
- **Développeurs** (création de l'application)
- **Logisticiens** (tests et validation)
- **Direction** (prise de décision)

Caractéristiques majeures

- **Application web responsive**, accessible sur PC et tablette.
- **Tableau de bord intuitif** avec indicateurs clés (taux de rupture, stock critique).
- **Système d'alerte automatique** quand un produit passe sous le seuil critique.

Contraintes et risques

- **Contraintes** : Déploiement en 6 mois, budget limité (10 000 000 Ar).
- **Risques** : Résistance au changement (employés habitués à Excel), intégration à l'ERP.

Livrable final

Un **cahier des charges détaillé** listant :

- Les **spécifications fonctionnelles** (gestion des stocks, alertes).
- Les **contraintes techniques** (base de données, compatibilité ERP).
- Les **tests utilisateurs** à prévoir avant mise en production.

3. Cahier des Charges Fonctionnel

3.1. Objectifs du Système

- Suivi **centralisé et en temps réel** des stocks.
- Réduction des erreurs grâce à **l'automatisation** des mises à jour.
- Alerte automatique pour éviter les ruptures de stock.
- Interface utilisateur simple et intuitive accessible sur **PC, tablette et mobile**.

3.2. Périmètre Fonctionnel

Fonctionnalités incluses

- **Gestion des stocks** : suivi des entrées/sorties en temps réel.
- **Alerte automatique** : notification en cas de stock faible.
- **Tableau de bord** : suivi des stocks, tendances des ventes.
- **Gestion des utilisateurs** : rôles et accès sécurisés.
- **Intégration ERP** : mise à jour automatique des stocks.

Hors périmètre

- Gestion comptable et financière.
- Suivi des fournisseurs externes.

3.3. Spécifications Fonctionnelles

1. Gestion des Stocks

- Interface permettant de **consulter les stocks en temps réel**.
- Possibilité de **filtrer par entrepôt, magasin, produit**.
- Mise à jour **automatique** après chaque vente ou réapprovisionnement.

2. Alerte Automatique

- Notification dès qu'un article passe sous un **seuil critique** (ex : 5 unités restantes).
- Envoi d'un **e-mail automatique** au responsable logistique.
- Possibilité de **définir des seuils personnalisés** pour chaque produit.

3. Gestion des Utilisateurs

3 profils utilisateurs :

- **Administrateur** (gestion complète)
- **Gestionnaire de stocks** (mise à jour des stocks)
- **Consultant** (lecture seule)

4. Tableau de Bord & Reporting

- Graphiques montrant **tendances des ventes, taux de rupture, stock critique**.
- Rapport exportable en **Excel/PDF**.
- Possibilité d'afficher les **statistiques par période** (jour/semaine/mois).

5. Intégration avec l'ERP

- Synchronisation automatique des données avec l'ERP de l'entreprise.
- Export des mouvements de stocks vers l'ERP.
- API REST pour accès aux données.

3.4. Contraintes et Risques

Type	Détails
Contraintes techniques	Doit être compatible avec le système actuel (Windows/Linux).
Contraintes budgétaires	Budget limité à 10 000 000 Ar , développement en 6 mois .
Risques	Résistance des employés au changement, intégration ERP complexe.

3.5. Technologies Préconisées

- **Frontend** : React.js (interface web intuitive)
- **Backend** : Node.js / Python (API robuste)
- **Base de données** : PostgreSQL / MySQL (gestion efficace des stocks)
- **Hébergement** : Cloud AWS/Azure (haute disponibilité)

4. Conclusion

L'étape "Délimiter" a permis de fixer un cadre clair, d'impliquer les parties prenantes et d'éviter les dérives en définissant **un périmètre précis**. L'entreprise peut maintenant passer aux étapes suivantes : conception, développement et déploiement.

Étape "Structurer"

L'objectif de cette étape est d'organiser et de structurer les **produits** (livrables), **travaux** et **tâches** nécessaires à la réalisation du projet selon trois axes :

1. **Composition** (relation "se compose de")
2. **Temps** (relation "devient" ou "s'enrichit de")
3. **Destinataires** (relation "est destiné à")

Ensuite, nous organiserons les **travaux** et **tâches** en **phases**, en incluant les **contrôles et suivis** associés.

1. Structuration des Produits

1.1 Identification des Produits

Type de Produit	Exemple Concret
Mis à la disposition de l'utilisateur	Interface web et mobile de gestion des stocks
Visibles par l'utilisateur	Manuel utilisateur, tutoriels vidéo
Nécessaires à l'utilisateur	Formation des employés, support technique
Utiles à l'environnement de travail	Environnement de test, outils de monitoring
Exigés par la méthode	Plan qualité, fiches projet, protocoles de test
Demandés par les partenaires	Reporting des stocks, rapports d'avancement

1.2 Structuration selon les 3 Axes

1 Axe de Composition ("se compose de")

L'application de gestion des stocks se compose de plusieurs **modules** :

- **Interface utilisateur**
 - Page de gestion des stocks
 - Tableaux de bord
 - Notifications & alertes
 - Formulaire de commande
- **Backend & API**
 - Gestion des bases de données
 - Connexion à l'ERP
 - API REST
- **Produits annexes**
 - Manuel utilisateur
 - Formation interne
 - Outils de reporting

2 Axe Temporel ("devient" ou "s'enrichit de")

Évolution de l'application dans le temps :

- **Version 1.0** : Suivi des stocks + alertes
- **Version 1.1** : Intégration avec l'ERP
- **Version 2.0** : Ajout du reporting avancé
- **Version 3.0** : Intelligence artificielle pour recommandations de commandes

3 Axe Destinataires ("est destiné à")

Produit	Destinataires
Interface utilisateur	Gestionnaires de stocks, responsables logistiques
API d'intégration ERP	Développeurs ERP
Tableaux de bord et reporting	Direction générale
Formation et support	Employés, administrateurs

2. Structuration des Travaux et des Tâches

2.1 Organisation des Travaux en Phases

Phases principales du projet :

1. **Conception**
 - Définition des besoins
 - Élaboration du cahier des charges
 - Conception UI/UX
2. **Développement**
 - Développement du backend
 - Création de l'interface utilisateur
 - Intégration avec l'ERP
3. **Tests et Validation**
 - Tests unitaires et fonctionnels
 - Tests avec utilisateurs finaux
4. **Déploiement et Formation**
 - Mise en production
 - Formation des utilisateurs
5. **Suivi et Maintenance**
 - Corrections et mises à jour

2.2 Ajout des Tâches de Début, Fin, et Contrôle

Exemples de tâches associées aux phases :

Phase	Tâches de début	Tâches de fin	Tâches de contrôle
Conception	Lancement du projet, recueil des besoins	Validation du cahier des charges	Vérification du cahier des charges

Développement	Définition de l'architecture technique	Finalisation du code	Revue de code, tests unitaires
Tests	Planification des scénarios de test	Validation des corrections	Tests de performance et sécurité
Déploiement	Préparation des serveurs	Mise en ligne	Vérification du bon fonctionnement
Suivi	Collecte des retours utilisateurs	Versioning et maintenance	Rapport d'amélioration

2.3 Présentation des Résultats

1 Liste des Produits et Travaux

- Application de gestion des stocks
- API d'intégration ERP
- Tableaux de bord
- Documentation utilisateur
- Rapports de suivi

2 Arborescence des Produits

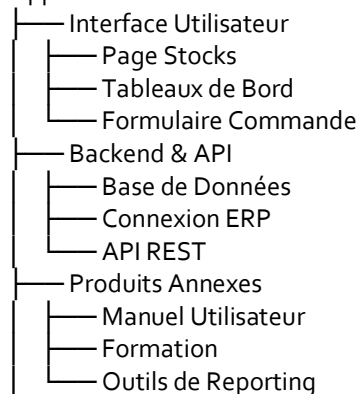
Application de gestion des stocks

- Interface utilisateur
 - Page de gestion des stocks
 - Tableaux de bord
- Backend
 - Connexion API
 - Gestion des bases de données

3 Diagramme de Structuration

Exemple de diagramme simplifié

1. Application de Gestion des Stocks



Conclusion

Grâce à l'étape "Structurer" :

- Les **produits** sont clairement identifiés et organisés.
- Les **travaux** sont bien **segmentés en phases et tâches**.
- Les livrables sont **associés aux bons destinataires**.
- L'ensemble du projet est **visualisé sous forme d'arborescence et de diagramme**.

Etape « Estimer »

Étude de Cas : Développement d'une Application de Gestion des Stocks

Nous allons appliquer **trois méthodes d'estimation** :

1. **COCOMO**
2. **Points de Fonction**
3. **Les Résultats Demandés**

Le but est d'estimer **la charge de travail, le délai et le coût du projet**.

Contexte du Projet

Une entreprise souhaite développer une **application web** de **gestion des stocks** permettant :

- **Suivi des entrées et sorties** de produits
- **Alertes de stock faible**
- **Gestion des fournisseurs et des commandes**
- **Tableaux de bord et reporting**
- **Exportation des données en Excel/PDF**

Effectif disponible :

- 4 développeurs
- 1 chef de projet
- 1 designer UI/UX

Technologies utilisées :

- **Front-end** : Angular
- **Back-end** : Java Spring Boot
- **Base de données** : MySQL

Contraintes :

- **Livraison en 6 mois**
- **Respect d'un budget maximal de 70 000 UM**

1. Estimation par la Méthode COCOMO

Nous supposons que l'application représente **18 KISL** (milliers d'instructions source livrées).

Formules COCOMO

- Charge (mois-homme) = $2,4 \times KISL^{1,05}$
- Temps (mois) = $2,5 \times Charge^{0,38}$

Calcul

- Charge brute = $2,4 \times (18)^{1,05} \approx 47$ mois-homme
- Temps estimé = $2,5 \times (47)^{0,38} \approx 9$ mois

Facteurs d'Ajustement

Les paramètres influençant la charge sont :

- Utilisation de frameworks modernes (-5%)
- Expérience des développeurs (-10%)
- Intégration avec des systèmes externes (export Excel/PDF) (+10%)
- Besoins de tests automatisés (+15%)

Facteur global d'ajustement = +10%

- Charge nette = $47 \times 1,1 \approx 52$ mois-homme

Répartition des Ressources

- 4 développeurs : 44 mois-homme
- Chef de projet : 5 mois-homme
- Designer UI/UX : 3 mois-homme

Livraison possible en 6 mois avec 4 développeurs.

Estimation du Coût

- Coût journalier d'un développeur = 450 UM/jour
- 52 mois-homme ≈ 1040 jours-homme
- Coût total = $1040 \times 450 = 468\ 000$ UM

Problème : Dépasse largement le budget !

Solution possible : Automatiser certaines tâches, réduire les fonctionnalités, ou externaliser certaines parties.

2. Estimation avec la Méthode des Points de Fonction

Type de Composant	Complexité	Nombre	Poids	Total
-------------------	------------	--------	-------	-------

Entrées (ajout de produits, commandes, fournisseurs)	Moyenne	5	4	20
Sorties (rapports, alertes, exports)	Moyenne	4	5	20
Interrogations (recherche de stock, historique des commandes)	Moyenne	3	4	12
Fichiers de Données Internes (produits, fournisseurs, transactions)	Moyenne	3	10	30
Fichiers de Données Externes (exports Excel/PDF)	Complexe	2	10	20
Total				102 PF

Facteur d'ajustement : +20% (complexité d'intégration et exportation des données)

Nombre de points ajustés = $102 \times 1,2 = 122,4$ PF

Calcul de la Charge de Travail

En se basant sur une conversion standard :

- 1 PF $\approx$ 8 heures de travail
- Charge = $122,4 \times 8 = 979$ heures $\approx$ 122 jours-homme

Coût estimé

- 122 jours-homme $\times$ 450 UM/jour = 54 900 UM
Solution plus réaliste en fonction du budget client

3. Estimation avec la Méthode des Résultats Demandés

Calcul des unités de programmation standard

- Nombre de résultats demandés (écrans, états, fichiers, messages) : 12
- Nombre de fichiers logiques : 4
- $N = (12 + 4) \times 1,5 = 24$ unités

Charges de Travail

- Étude technique : $(10 + 24 / 5) \times 2 = 19$ j-h
- Dossiers de programme : $2 \times 24 = 48$ j-h
- Réalisation et tests : $5 \times 24 = 120$ j-h
- Intégration et recette : $(10 + 24 / 10) \times 2 = 14,8$ j-h $\approx$ 15 j-h
Total = $19 + 48 + 120 + 15 = 202$ jours-homme

Coût estimé

- $202 \text{ jours-homme} \times 450 \text{ UM/jour} = 90\,900 \text{ UM}$
- **Problème** : Dépasse le budget client.
- **Réduction des charges** en simplifiant les interfaces et optimisant les tests.

Comparaison des Méthodes

Méthode	Charge (jours-homme)	Coût Estimé (UM)
COCOMO	1040	468 000
Points de Fonction	122	54 900
Résultats Demandés	202	90 900

Conclusion

- **COCOMO** surestime la charge, plus adapté aux très grands projets.
- **Points de Fonction** est plus précis et respecte le budget.
- **Résultats Demandés** est utile mais donne une estimation trop élevée.

Solution Recommandée

Utiliser la méthode **Points de Fonction**, qui est plus réaliste et permet d'adapter le projet au budget disponible.

Travail à Faire pour les Étudiants

1. Refaire l'estimation avec d'autres hypothèses (ex. : moins de fonctionnalités).
2. Comparer les méthodes et proposer des améliorations.
3. Optimiser la charge pour réduire les coûts.